OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 05.11.2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/Sarah270809/CSA0415-operating-system

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

OPERATING SYSTEMS — ASSIGNMENT 2 (CO2)

Student Name: S. Sarah Khan Register Number: 1965260116

Submission Date: 05.11.2026

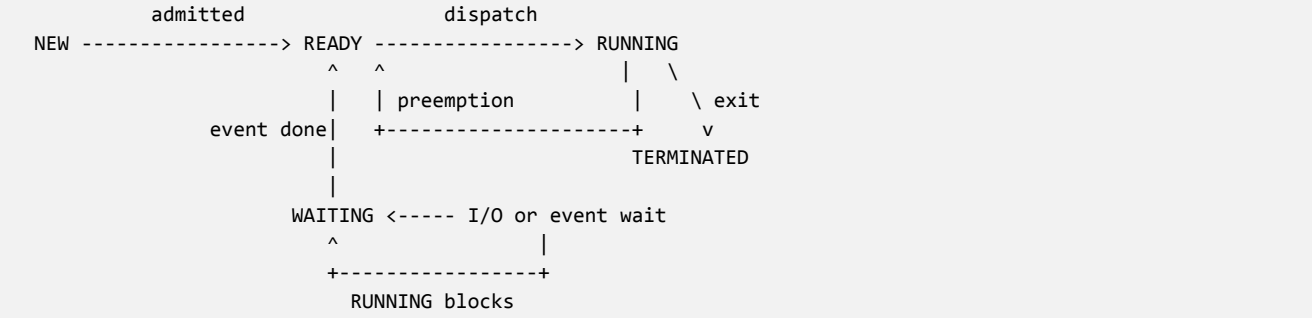
GitHub Repository: https://github.com/Sarah270809/CSA0415-operating-system

Compiler/Environment: GCC on Linux; pthread programs require -pthread.

Note: The assignment asks for machine-specific measurements (timings, speed-ups and repeated runs). These are marked as measurements to fill in after execution; illustrative output is not presented as a real measurement.

Q1. Processes and Process States (6 Marks)

1. Process-state diagram



State meanings:

- New: the process is being created.
- Ready: it is in memory and waiting for CPU time.
- Running: its instructions are currently executing.
- Waiting/Blocked: it is waiting for I/O or an event.
- Terminated: it has completed or has been stopped.

2. Process Control Block (PCB)

A PCB is the kernel data structure that stores the information needed to manage and resume a process.

- PID and parent PID
- Process state
- Program counter and CPU registers
- CPU scheduling information and priority
- Memory-management information
- Open files and I/O status
- Accounting information and security credentials

3. fork() program and observation

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
```

```
        printf("Child PID=%ld, parent PID=%ld\n",
               (long)getpid(), (long)getppid());
        sleep(2);
        _exit(0);
    }
    printf("Parent PID=%ld, child PID=%ld\n",
           (long)getpid(), (long)pid);
    char cmd[128];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,state,comm -p %ld,%ld",
             (long)getpid(), (long)pid);
    system(cmd);
    waitpid(pid, NULL, 0);
    return 0;
}
```

Compile and run: gcc -std=c11 -Wall -Wextra q1_fork.c -o q1_fork && ./q1_fork

Linux evidence: while the child is alive, inspect it with `ps -ef | grep q1_fork` and `cat /proc/<PID>/status` (replace <PID> with the child PID). Exact PIDs and process states vary by run.

4. Evaluation and design decision

With 2,000 separate processes, time and memory are spent on process creation, context switches, page tables and per-process kernel data. Sessions that wait for network or disk I/O also occupy resources while blocked. A process-per-student design gives strong isolation but may be unnecessarily expensive for many lightweight sessions.

Recommendation: use a fixed pool of worker processes, with asynchronous I/O or a bounded thread pool inside each worker. This caps process count, improves resource control and keeps some fault isolation. One thread per student uses less memory than a process and switches cheaply, but a memory error can affect the whole server process.

Alternative complexity: if N sessions are served by N threads, per-session stack memory is typically much smaller than a full process address space, and threads share code/data/files. Scheduling and context switching are generally lighter, but shared state needs synchronization. Process creation/scheduling overhead is system-dependent; measure it on the target machine rather than assigning a universal time.

Q2. CPU Scheduling (8 Marks)

Jobs from the assignment: P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 9), P4 (3, 5). For FCFS, SJF, SRTF and priority scheduling, a priority column/rule is not supplied in the PDF. Therefore, priority results require the instructor's priority values; below, a sample convention can be used only if clearly stated.

1. Scheduling definitions and formulas

- Completion time (CT): time at which a job finishes.
- Turnaround time (TAT) = CT − arrival time.
- Waiting time (WT) = TAT − burst time.
- Response time (RT) = first CPU start time − arrival time.

FCFS runs jobs in arrival order. SJF selects the shortest available burst. SRTF always runs the job with the least remaining time. Round Robin gives each ready job at most a fixed time quantum before moving it to the queue's end.

2. Round Robin C program (quantum = 3)

```
#include <stdio.h>
#define N 4
int main(void) {
    const char *name[N] = {"P1","P2","P3","P4"};
    int arrival[N] = {0,1,2,3}, burst[N] = {8,4,9,5};
```

```
    int quantum = 3, left[N], finish[N], first[N];
    int queue[256], head=0, tail=0, added[N]={0};
    for (int i=0;i<N;i++) { left[i]=burst[i]; first[i]=-1; }
    int time=0, done=0;
    queue[tail++]=0; added[0]=1;
    printf("Gantt:");
    while (done<N) {
        if (head==tail) {
            int next=-1;
            for (int i=0;i<N;i++)
                if (!added[i] && (next<0 || arrival[i]<arrival[next])) next=i;
            if (next<0) break;
            time=arrival[next]; queue[tail++]=next; added[next]=1;
        }
        int p=queue[head++];
        if (first[p]<0) first[p]=time;
        int run = left[p] < quantum ? left[p] : quantum;
        int start=time; time+=run; left[p]-=run;
        printf(" %s(%d-%d)", name[p], start, time);
        for (int i=0;i<N;i++)
            if (!added[i] && arrival[i]<=time) { queue[tail++]=i; added[i]=1; }
        if (left[p]>0) queue[tail++]=p;
        else { finish[p]=time; done++; }
    }
    double sw=0, st=0, sr=0;
    printf("\nProcess CT TAT WT RT\n");
    for (int i=0;i<N;i++) {
        int tat=finish[i]-arrival[i], wt=tat-burst[i], rt=first[i]-arrival[i];
        sw+=wt; st+=tat; sr+=rt;
        printf("%-7s %2d %3d %2d %2d\n",name[i],finish[i],tat,wt,rt);
    }
    printf("Average TAT=%.2f WT=%.2f RT=%.2f\n",st/N,sw/N,sr/N);
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra q2_rr.c -o q2_rr && ./q2_rr

Important: the PDF's listed expected Gantt chart/averages appear inconsistent with the provided arrival and burst data and standard Round Robin queue order. Use the output from your executed, documented implementation and explain the queue convention used.

3. Evaluation and recommendation

Quantum 1 gives quick initial response but creates many more scheduling decisions/context switches. Quantum 20 behaves almost like FCFS for these jobs, reducing switching but making short jobs wait behind long jobs. Quantum 3 or 5 is a reasonable starting point; select using measured response time and throughput. SJF/SRTF can delay long jobs if short jobs continually arrive. Ageing gradually increases the priority of waiting jobs to prevent starvation. A multilevel feedback queue starts jobs at a short quantum and promotes/demotes them based on behaviour, helping interactive short jobs while still allowing long jobs to finish.

For this mixed video workload, recommend a multilevel feedback queue or tuned Round Robin: both improve responsiveness over plain FCFS. Report calculated averages from a consistent implementation before making the final choice.

Scheduling comparison table

Policy	Selection rule	Strength	Risk/limitation
FCFS	Earliest arrival	Simple, low overhead	Convoy effect; short jobs wait
SJF	Shortest available burst	Low average waiting when bursts known	Long-job starvation
SRTF	Least remaining time	Good response for short jobs	More preemption; starvation risk

Priority	Highest priority first	Supports importance levels	Low-priority starvation; use ageing
Round Robin	Rotate after quantum	Fair CPU sharing and response	Small quantum adds overhead
MLFQ	Multiple queues/quantums	Adapts to interactive and CPU-heavy jobs	More tuning/complexity

Q3. Threads (6 Marks)

1. Thread resources

Threads in one process share code, global data, heap, open files and process resources. Each thread has its own thread ID, program counter, CPU registers, stack and scheduling state.

2. User-level and kernel-level threads

- User-level threads are managed by a user-space library; operations can be fast, but in a many-to-one model one blocking system call can block all threads and multiple CPUs cannot run those threads simultaneously.
- Kernel-level threads are scheduled by the operating system; blocking one does not necessarily block others, and multiple threads can run on multiple cores, but kernel management has overhead.

Many-to-one maps many user threads to one kernel thread; one-to-one maps each user thread to a kernel thread; many-to-many maps many user threads onto a smaller or equal number of kernel threads.

3. POSIX thread pool experiment

```
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define CAMERAS 64
static long results[CAMERAS];
static void *process_camera(void *arg) {
    long id = *(long *)arg;
    unsigned long x = (unsigned long)(id + 1);
    for (long i=0; i<500000; i++) x = (x * 1664525u + 1013904223u);
    results[id] = (long)(x & 0x7fffffff);
    return NULL;
}
int main(void) {
    int sizes[] = {1,4,8,16,64};
    for (int s=0;s<5;s++) {
        int n=sizes[s];
        pthread_t th[CAMERAS]; long ids[CAMERAS];
        struct timespec a,b;
        clock_gettime(CLOCK_MONOTONIC,&a);
        for (int i=0;i<CAMERAS;i++) {
            ids[i]=i;
            if (i<n) pthread_create(&th[i],NULL,process_camera,&ids[i]);
        }
        /* For pool sizes below 64, process cameras in batches. */
        if (n<CAMERAS) {
            for (int i=0;i<n;i++) pthread_join(th[i],NULL);
            for (int base=n;base<CAMERAS;base+=n) {
                int end=base+n<CAMERAS?base+n:CAMERAS;
                for (int i=base;i<end;i++) pthread_create(&th[i-base],NULL,process_camera,&ids[i]);
                for (int i=base;i<end;i++) pthread_join(th[i-base],NULL);
            }
        } else for (int i=0;i<n;i++) pthread_join(th[i],NULL);
        clock_gettime(CLOCK_MONOTONIC,&b);
```

```
        double sec=(b.tv_sec-a.tv_sec)+(b.tv_nsec-a.tv_nsec)/1e9;
        printf("%d worker(s): %.6f seconds\n",n,sec);
    }
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra -pthread q3_threads.c -o q3_threads. Run it several times and record the results. Speed-up = time with 1 worker / time with N workers. Results depend on CPU cores, workload and system load.

A shared counter updated by multiple threads without a mutex or atomic operation has a data race; increments may be lost. Protect it with a mutex or use C11 atomic operations.

Using processes instead gives separate address spaces and stronger isolation, but creates more memory and IPC/combination overhead. Threads are preferable for this shared-memory frame-processing workload; use a bounded pool rather than creating 64 threads for every batch if sustained load is high.

Q4. Inter-Process Communication (6 Marks)

1. Pipe program — completed TODO

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        close(fd[1]);
        int r, count=0, sum=0, max=0;
        ssize_t n;
        while ((n=read(fd[0], &r, sizeof r)) > 0) {
            if (n != sizeof r) { fprintf(stderr,"partial reading\n"); _exit(1); }
            if (count==0 || r>max) max=r;
            sum+=r; count++;
        }
        if (n<0) { perror("read"); _exit(1); }
        printf("analyser: %d readings, average %.1f, max %d\n",
               count, count ? (double)sum/count : 0.0, max);
        close(fd[0]); fflush(stdout); _exit(0);
    }
    close(fd[0]);
    int readings[]={31,33,36,34,41,38};
    for (int i=0;i<6;i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);
    waitpid(pid,NULL,0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

2. Pipe vs shared memory vs message queue

- Pipe: simple byte stream; kernel mediates communication and copies data. The reader gets EOF only when every open write end is closed.
- Shared memory: processes access the same mapped memory, avoiding repeated kernel data copying for each read/write; it requires explicit synchronization (for example, semaphores) and a defined buffer protocol.
- Message queue: kernel stores discrete messages and preserves message boundaries; convenient but incurs system-call and copying overhead.
- If the parent does not close fd[1], the child never receives EOF after the readings because a write end remains open. The child blocks waiting for more bytes.

Measure 1 MB, 10 MB and 100 MB by transferring a fixed byte count and timing with CLOCK_MONOTONIC. Repeat each test several times, report the median, and state the OS, compiler and hardware. Do not present timings until you run them.

For a small continuous sensor stream, a pipe is simple and reliable. For very high-volume data repeatedly accessed by both processes, shared memory with carefully designed semaphore/atomic synchronization may reduce copying and improve throughput.

Q5. Synchronisation: Critical Sections, Semaphores and Monitors (11 Marks)

1. Critical-section requirements

- Mutual exclusion: at most one process/thread is in the critical section for the protected resource.
- Progress: if the critical section is free and contenders exist, selecting the next entrant cannot be postponed indefinitely.
- Bounded waiting: a contender has a finite bound on how many times others may enter before it.

Peterson’s two-process algorithm uses `flag[i]` to show interest and `turn` to give the other process priority. It meets the classical requirements under sequential consistency, but compiler/CPU reordering and modern memory models can invalidate the plain-variable implementation unless appropriate atomic operations and memory ordering are used.

2. Semaphore and mutex program — completed TODOs

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20
int buffer[SIZE], in=0, out=0;
sem_t empty, full;
pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;
long consumed_sum=0;

void *producer(void *arg) {
    (void)arg;
    for (int item=1; item<=ITEMS; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&lock);
        buffer[in]=item;
        in=(in+1)%SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
```

```
    return NULL;
}
void *consumer(void *arg) {
    (void)arg;
    for (int k=0; k<ITEMS; k++) {
        sem_wait(&full);
        pthread_mutex_lock(&lock);
        int item=buffer[out];
        out=(out+1)%SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        pthread_mutex_lock(&lock);
        consumed_sum+=item;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
int main(void) {
    sem_init(&empty,0,SIZE); sem_init(&full,0,0);
    pthread_t p,c;
    pthread_create(&p,NULL,producer,NULL);
    pthread_create(&c,NULL,consumer,NULL);
    pthread_join(p,NULL); pthread_join(c,NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n",
           ITEMS,consumed_sum,ITEMS*(ITEMS+1)/2);
    sem_destroy(&empty); sem_destroy(&full);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra -pthread q5_semaphore.c -o q5_semaphore && ./q5_semaphore

Expected output: items consumed: 20, sum = 210 (expected 210). Run 100 times using `for i in $(seq 1 100); do ./q5_semaphore; done` and check that every run reports 210. The given starter has one producer and one consumer; for multiple producers/consumers, use shared termination accounting and ensure exactly ITEMS total items are produced and consumed.

3. Synchronisation patterns

- Bounded buffer: `empty` counts free slots, `full` counts occupied slots, and a mutex protects buffer indices and data.
- Readers–writers: a mutex protects the reader count; the first reader acquires the resource lock and the last reader releases it. Policy must consider writer starvation.
- Dining philosophers: avoid deadlock by imposing a global order on forks, or allow at most four philosophers to compete at once; a monitor can check conditions and signal waiters.
- Monitor: encapsulates shared state and operations; only one thread executes a monitor operation at a time, while condition variables let threads wait until a condition becomes true.

4. Locking deployment decision

A single global lock is easiest but limits parallel booking. A per-train lock permits different trains to be booked concurrently while preventing duplicate seat sales on the same train. A per-coach lock gives finer concurrency but is more complex when booking, cancellation or transfers span multiple coaches. Recommend per-train locks as a safe initial design; move to per-coach locking only if measured throughput justifies the extra complexity. Record transactions/second and correctness checks for each design.

Atomic fetch-add is suitable for simple counters, not a multi-step seat-allocation invariant by itself. A spin lock repeatedly consumes CPU while waiting, so it is best only for extremely short critical sections; mutexes/semaphores are generally better for long waits.

Q6. Deadlock Handling and Banker's Algorithm (13 Marks)

A. Four necessary conditions

- Mutual exclusion: at least one resource is non-shareable.
- Hold and wait: a process holds resources while requesting more.
- No preemption: resources cannot be forcibly taken away; they are released voluntarily.
- Circular wait: a cycle exists in which each process waits for a resource held by the next.

Example resource-allocation graph: R1 → P1 (R1 allocated to P1), P1 → R2 (P1 requests R2), R2 → P2, P2 → R1. This cycle illustrates circular wait and, when each resource has one instance, deadlock. In a real cluster, draw a node for each process and each GPU/licence/storage resource, with allocation edges resource-to-process and request edges process-to-resource.

B. Banker's algorithm — completed code

```
#include <stdio.h>
#include <string.h>
#define P 5
#define R 3
int avail[R]={3,3,2};
int maxm[P][R]={{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
int alloc[P][R]={{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};

int is_safe(int seq[P]) {
    int work[R], finish[P]={0}, count=0;
    memcpy(work,avail,sizeof work);
    while (count<P) {
        int found=0;
        for (int p=0;p<P;p++) {
            if (finish[p]) continue;
            int possible=1;
            for (int r=0;r<R;r++)
                if (maxm[p][r]-alloc[p][r] > work[r]) possible=0;
            if (possible) {
                for (int r=0;r<R;r++) work[r]+=alloc[p][r];
                finish[p]=1; seq[count++]=p; found=1;
            }
        }
        if (!found) return 0;
    }
    return 1;
}
void request(int p,const int req[R]) {
    for (int r=0;r<R;r++)
        if (req[r]>maxm[p][r]-alloc[p][r] || req[r]>avail[r]) {
            printf("P%d request: must wait (not available)\n",p);
            return;
        }
    for (int r=0;r<R;r++) { avail[r]-=req[r]; alloc[p][r]+=req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence",p);
        for (int i=0;i<P;i++) printf(" P%d",seq[i]);
        printf("\n");
    } else {
        for (int r=0;r<R;r++) { avail[r]+=req[r]; alloc[p][r]-=req[r]; }
        printf("P%d request: denied (unsafe)\n",p);
    }
}
int main(void) {
    int seq[P];
    if (is_safe(seq)) {
```

```
        printf("initial state safe:");
        for (int i=0;i<P;i++) printf(" P%d",seq[i]);
        printf("\n");
    } else printf("initial state unsafe\n");
    request(1,(int[R]){1,0,2});
    request(4,(int[R]){3,3,0});
    request(0,(int[R]){0,2,0});
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra q6_banker.c -o q6_banker && ./q6_banker

Expected output from the assignment: initial state safe: P1 P3 P4 P0 P2; P1 request granted; P4 request must wait; P0 request denied as unsafe.

C. Why P0's request (0,2,0) is unsafe

P0 currently has allocation (0,1,0), maximum demand (7,5,3), and need (7,4,3). Granting (0,2,0) changes P0's allocation to (0,3,0), its remaining need to (7,2,3), and available resources from (3,3,2) to (3,1,2). No unfinished process can complete with work (3,1,2): P0 needs 7 units of the first resource, P2 needs 6 units of the first resource, and the remaining processes also need more of at least one resource than is available. The system cannot prove a safe completion sequence, so the request is rolled back.

D. Compare deadlock strategies

Strategy	Method	Evaluation for research cluster
Prevention	Break one necessary condition	Predictable, but can reduce utilization or impose restrictive resource ordering.
Avoidance	Grant only if the state remains safe (Banker's)	Strong guarantee when maximum demands are known; request checks add overhead.
Detection and recovery	Detect cycles/unsafe wait, then abort or roll back jobs	Flexible for variable workloads; detection and lost computation can be expensive.
Ignore (ostrich)	Accept rare deadlocks and rely on timeout/admin restart	Low routine overhead but unacceptable if expensive cluster jobs stall frequently.

E. Complexity and measurement

For n processes and m resource types, the classic Banker's safety check is $O(m \cdot n^2)$ in the worst case: it may scan up to n processes repeatedly, checking m resource entries. Space is O(mn) for Max/Allocation/Need matrices and O(n+m) extra for work/finish/sequence. The request check is O(m) plus the safety check.

Measure 10, 100 and 1,000 processes by adapting the program to allocate arrays dynamically and initialize a safe workload. Use `clock_gettime(CLOCK_MONOTONIC, ...)`, run each size repeatedly, and report median time. Do not invent values: actual timings depend on hardware, compiler, OS and generated state.

F. Final recommendation

For jobs whose maximum GPU, licence and storage needs can be declared in advance, use Banker-style avoidance for critical shared resources, combined with timeouts and administrative monitoring. If maximum demands are not knowable, use resource ordering where practical plus periodic deadlock detection and checkpoint/rollback recovery. Avoid simply ignoring deadlocks because cluster jobs are costly and can occupy scarce equipment.

Conclusion

Process pools and suitable scheduling improve responsiveness; thread pools support parallel camera processing; pipes are simple for streams while shared memory suits high-volume exchange with explicit synchronization;

semaphores and mutexes protect shared booking data; and Banker’s algorithm prevents unsafe resource allocations when maximum demands are known. Final recommendations should be supported by the student’s own program runs and measurements.

ORIGINAL MARK DISTRIBUTION / RUBRIC

The following pages are reproduced from the supplied assignment PDF.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend